UD3 - Consultas SQL (V)

Nesta última parte da unidade didáctica estudaremos como combinar os resultados de varias consultas mediante `UNION` e `UNION ALL`. Tamén aprenderemos a mellorar o rendemento das consultas SQL, utilizar índices e interpretar os plans de execución mediante `EXPLAIN`.

23. Combinación de seleccións con UNION e UNION ALL

23.1. Que é UNION?

O operador `UNION` permite combinar os resultados de dúas ou máis instrucións `SELECT` nun único resultado.

A diferenza dos `JOIN`, que combinan columnas de diferentes táboas nunha mesma fila, `UNION` engade os rexistros dunha consulta aos rexistros doutra.

> **Idea clave:** `JOIN` combina información horizontalmente, incorporando columnas de varias táboas. `UNION` combina resultados verticalmente, engadindo filas dunha consulta ás doutra.

23.2. Sintaxe

```
SELECT columnas
FROM taboa1
WHERE condicion

UNION

SELECT columnas
FROM taboa2
WHERE condicion;
```

Cada consulta debe devolver o mesmo número de columnas e as columnas correspondentes deben ter tipos de datos compatibles.

23.3. Compatibilidade entre os resultados

Para combinar varias consultas mediante `UNION`, débense cumprir as seguintes condicións:

- Todas as consultas deben devolver o mesmo número de columnas.

- As columnas deben aparecer na mesma orde.
- Os tipos de datos das columnas correspondentes deben ser compatibles.
- Os nomes das columnas do resultado final proceden da primeira consulta.

Non é necesario que as columnas teñan exactamente o mesmo nome, pero si deben representar información compatible.

23.4. Exemplo: combinar nomes de alumnos e profesores

Supoñamos que temos dúas táboas:

- `alumnos` : contén os nomes dos alumnos.
- `profesores` : contén os nomes dos profesores.

Queremos obter unha lista única cos nomes das persoas que aparecen en ambas as táboas.

```
SELECT nombre
FROM alumnos

UNION

SELECT nombre
FROM profesores;
```

O resultado inclúe os nomes das dúas táboas. Se un mesmo nome aparece varias veces, `UNION` elimina os rexistros duplicados.

23.5. Uso de UNION ALL

O operador `UNION ALL` tamén combina os resultados de varias consultas, pero conserva todos os rexistros, incluídos os duplicados.

```
SELECT nombre
FROM alumnos

UNION ALL

SELECT nombre
FROM profesores;
```

Neste caso, se unha persoa aparece nas dúas táboas ou un nome está repetido dentro dunha delas, aparecerá tantas veces como figure nos resultados.

23.6. Diferenza entre UNION e UNION ALL

Operador	Resultado	Tratamento dos duplicados
`UNION`	Combina os resultados.	Elimina as filas duplicadas.
`UNION ALL`	Combina os resultados.	Conserva as filas duplicadas.

23.7. Combinar consultas con condicións

Cada instrución `SELECT` pode incluír as súas propias condicións de filtrado.

Por exemplo, queremos obter os nomes dos alumnos maiores de idade e os nomes dos profesores:

```
SELECT nombre
FROM alumnos
WHERE edad >= 18

UNION

SELECT nombre
FROM profesores;
```

A primeira consulta selecciona os alumnos maiores de idade e a segunda recupera todos os profesores. Finalmente, `UNION` combina ambos os resultados eliminando duplicados.

23.8. Combinar máis de dúas consultas

Podemos utilizar varios operadores `UNION` para combinar os resultados de tres ou máis consultas.

```
SELECT nombre
FROM alumnos

UNION

SELECT nombre
FROM profesores

UNION

SELECT nombre
FROM cursos;
```

Para que esta combinación sexa válida, as tres consultas deben devolver unha columna compatible.

23.9. Exercicios de consolidación

1. Combina os nomes dos alumnos e dos profesores utilizando `UNION`.
2. Repite a consulta anterior utilizando `UNION ALL`.
3. Combina os alumnos maiores de idade cos profesores.
4. Combina os identificadores e nomes dos alumnos e profesores, sempre que as columnas sexan compatibles.
5. Realiza unha combinación de tres consultas utilizando `UNION ALL`.

24. UNION, UNION ALL e ordenación dos resultados

24.1. Como se eliminan os duplicados?

Cando utilizamos `UNION`, MariaDB elimina as filas duplicadas do resultado combinado.

Considéranse duplicadas as filas nas que coinciden os valores de todas as columnas devoltas pola consulta.

Por exemplo:

```
SELECT nombre, edad
FROM alumnos

UNION

SELECT nombre, edad
FROM antiguos_alumnos;
```

Se unha persoa aparece nas dúas táboas co mesmo nome e idade, só aparecerá unha vez no resultado.

Pola contra, se coincide o nome pero non a idade, as filas considéranse diferentes e conservaranse ambas.

24.2. UNION ALL e rendemento

`UNION ALL` non necesita eliminar os duplicados, polo que normalmente require menos traballo que `UNION`.

Se sabemos que non existen duplicados ou queremos conservalos, é recomendable utilizar `UNION ALL`.

24.3. Ordenación dos resultados combinados

Podemos ordenar o resultado dunha combinación utilizando unha única cláusula `ORDER BY`, situada ao final de todas as consultas.

```
SELECT nombre
FROM alumnos

UNION

SELECT nombre
FROM profesores

ORDER BY nombre ASC;
```

A cláusula `ORDER BY` aplícase ao resultado completo, non a cada consulta individual.

24.4. Ordenación por varias columnas

Tamén podemos ordenar utilizando varias columnas do resultado combinado.

```
SELECT nombre, edad
FROM alumnos

UNION ALL

SELECT nombre, edad
FROM antiguos_alumnos

ORDER BY edad DESC, nombre ASC;
```

Primeiro ordénanse os rexistros pola idade de maior a menor e, en caso de empate, polo nome en orde alfabética.

24.5. Uso de alias no resultado

Os nomes das columnas do resultado combinado proceden da primeira consulta. Podemos definir alias nela e utilizalos posteriormente na ordenación.

```
SELECT nombre AS persoa, edad AS anos
FROM alumnos

UNION ALL

SELECT nombre, edad
FROM profesores

ORDER BY anos DESC;
```

Neste exemplo, o resultado terá as columnas `persoa` e `anos`.

24.6. Uso de LIMIT

Podemos limitar o número de filas devoltas polo resultado combinado utilizando `LIMIT`.

```
SELECT nombre
FROM alumnos

UNION

SELECT nombre
FROM profesores

ORDER BY nombre ASC
LIMIT 10;
```

A consulta devolve os dez primeiros nomes do resultado combinado, ordenados alfabeticamente.

24.7. Outras operacións de combinación

Ademais de `UNION`, existen outros operadores relacionais para combinar resultados de consultas:

- `UNION`: combina os rexistros de ambas as consultas e elimina duplicados.
- `UNION ALL`: combina os rexistros conservando duplicados.
- `INTERSECT`: devolve os rexistros que aparecen nos dous resultados.
- `EXCEPT`: devolve os rexistros da primeira consulta que non aparecen na segunda.

MariaDB incorpora os operadores `INTERSECT` e `EXCEPT` nas súas versións actuais. A súa dispoñibilidade pode depender da versión instalada.

24.8. Exemplo de INTERSECT

O operador `INTERSECT` permite obter os valores que aparecen nos resultados das dúas consultas.

```
SELECT nombre
FROM alumnos

INTERSECT

SELECT nombre
FROM antiguos_alumnos;
```

O resultado contén os nomes que aparecen nas dúas táboas. Os duplicados elimínanse, salvo que se utilice a variante `INTERSECT ALL` cando estea dispoñible.

24.9. Exemplo de EXCEPT

O operador `EXCEPT` devolve os rexistros da primeira consulta que non aparecen na segunda.

```
SELECT nombre
FROM alumnos

EXCEPT

SELECT nombre
FROM antiguos_alumnos;
```

O resultado contén os nomes dos alumnos que non aparecen na táboa de antigos alumnos.

24.10. Resumo das operacións

Operador	Funcionamento
`UNION`	Unión de resultados sen duplicados.
`UNION ALL`	Unión de resultados conservando duplicados.
`INTERSECT`	Rexistros comúns aos dous resultados.
`EXCEPT`	Rexistros do primeiro resultado que non están no segundo.

24.11. Exercicios de consolidación

1. Combina os nomes dos alumnos e profesores e ordena o resultado alfabeticamente.
2. Obtén os dez primeiros nomes do resultado dunha combinación.
3. Combina dúas consultas con `UNION ALL` e ordena por idade descendente.
4. Obtén os nomes que aparecen tanto na táboa de alumnos como na de antigos alumnos mediante `INTERSECT`.
5. Obtén os alumnos que non aparecen na táboa de antigos alumnos mediante `EXCEPT`.

25. Optimización de consultas SQL

25.1. Por que é necesario optimizar as consultas?

Cando unha base de datos almacena poucos rexistros, moitas consultas execútanse rapidamente. Porén, cando o volume de información aumenta, algunhas consultas poden tardar demasiado en devolver os resultados.

A optimización consiste en escribir e executar consultas de maneira que utilicen os recursos do sistema de forma eficiente.

Unha consulta optimizada pode reducir:

- O número de rexistros que é necesario examinar.
- A cantidade de datos que se deben recuperar.
- As operacións de ordenación e agrupamento.
- O tempo de execución e o consumo de recursos.

25.2. Seleccionar unicamente as columnas necesarias

Unha práctica básica de optimización consiste en evitar utilizar `SELECT *` cando non necesitamos todas as columnas dunha táboa.

Consulta pouco específica:

```
SELECT *
FROM alumnos;
```

Consulta máis eficiente cando só necesitamos dous campos:

```
SELECT nombre, edad
FROM alumnos;
```

Ao seleccionar unicamente as columnas necesarias reducimos a cantidade de información que debe ser recuperada e transferida.

25.3. Utilizar filtros adecuados

A cláusula `WHERE` permite limitar os rexistros que participan nunha consulta.

```
SELECT nombre, edad
FROM alumnos
WHERE edad >= 18;
```

Esta consulta só recupera os alumnos maiores de idade.

É recomendable aplicar os filtros necesarios e evitar recuperar grandes cantidades de datos que despois non se van utilizar.

25.4. Evitar funcións innecesarias nas condicións

Cando aplicamos unha función directamente sobre unha columna nunha condición, podemos dificultar o aproveitamento dun índice sobre esa columna.

Por exemplo:

```
SELECT *
FROM alumnos
WHERE YEAR(fecha_nacimiento) = 2005;
```

Se existe un índice sobre `fecha_nacimiento`, esta expresión pode impedir que se utilice directamente para localizar os rexistros.

Unha alternativa é expresar a condición mediante un intervalo:

```
SELECT *
FROM alumnos
WHERE fecha_nacimiento >= '2005-01-01'
  AND fecha_nacimiento < '2006-01-01';
```

Esta segunda forma permite que o motor poida aproveitar un índice sobre o campo `fecha_nacimiento`. O uso efectivo do índice dependerá do plan de execución.

25.5. Que é un índice?

Un índice é unha estrutura de datos que permite localizar rexistros dunha táboa de maneira máis rápida, evitando en determinados casos ter que examinar todos os rexistros.

Podemos comparalo co índice dun libro: en lugar de revisar todas as páxinas para localizar un tema, consultamos o índice para atopar directamente a súa localización.

> **Idea clave:** os índices poden mellorar as operacións de busca, filtrado e unión de táboas, pero tamén ocupan espazo e poden facer máis custosas as operacións de inserción, actualización e eliminación.

25.6. Crear índices

Podemos crear un índice sobre unha columna utilizando a instrución `CREATE INDEX`.

```
CREATE INDEX idx_alumnos_edad
ON alumnos(edad);
```

Este índice pode axudar nas consultas que filtran ou ordenan pola columna `edad`.

Para crear un índice sobre varias columnas:

```
CREATE INDEX idx_alumnos_curso_edad
ON alumnos(curso_id, edad);
```

Este índice composto pode resultar útil en consultas que utilizan ambas as columnas, especialmente cando filtran primeiro por `curso_id` e tamén utilizan `edad`.

25.7. Índices e claves

As claves primarias e as restricións de unicidade tamén utilizan índices para garantir a identificación dos rexistros e acelerar determinadas operacións.

- `PRIMARY KEY`: identifica de maneira única cada rexistro.
- `UNIQUE`: impide que se repitan determinados valores.

- `INDEX`: facilita a busca e recuperación de información.

As claves foráneas tamén poden beneficiarse de índices. En InnoDB, MariaDB require un índice adecuado para as columnas que forman unha clave foránea e pode crealo automaticamente se non existe outro índice válido.

25.8. Consultar os índices dunha táboa

Podemos consultar os índices existentes mediante `SHOW INDEX`.

```
SHOW INDEX FROM alumnos;
```

O resultado permite consultar información como:

- O nome do índice.
- A columna ou columnas incluídas.
- Se permite valores duplicados.
- A orde das columnas nun índice composto.

25.9. Eliminar índices

Se un índice xa non é necesario, podemos eliminalo mediante `DROP INDEX`.

```
DROP INDEX idx_alumnos_edad
ON alumnos;
```

Non debemos eliminar índices asociados a claves ou restricións sen comprobar antes as súas dependencias.

25.10. Boas prácticas de optimización

- Seleccionar só as columnas necesarias.
- Aplicar filtros adecuados mediante `WHERE`.
- Utilizar índices nas columnas que se consultan frecuentemente cando estea xustificado.
- Evitar crear índices innecesarios.
- Revisar as condicións de unión entre táboas.
- Evitar operacións e funcións innecesarias sobre as columnas utilizadas nos filtros.
- Analizar o plan de execución antes de modificar unha consulta.

25.11. Exercicios de consolidación

1. Consulta todos os alumnos utilizando `SELECT *` e despois modifica a consulta para recuperar só o nome e a idade.
2. Crea un índice sobre a columna `edad` da táboa `alumnos`.
3. Consulta os índices existentes na táboa.
4. Compara dúas consultas que filtren pola idade antes e despois de crear o índice.
5. Crea un índice composto sobre `curso_id` e `edad`.
6. Elimina o índice que creaches e comproba que desapareceu.

26. Análise do plan de execución con EXPLAIN

26.1. Que é EXPLAIN?

A instrución `EXPLAIN` permite analizar como o sistema xestor de bases de datos pretende executar unha consulta SQL.

O plan de execución ofrece información sobre:

- As táboas que participan na consulta.
- A orde na que se procesan.
- Os índices que se poderían utilizar.
- O tipo de acceso aos rexistros.
- O número estimado de filas que se deben examinar.

Esta información permite identificar posibles problemas de rendemento e decidir se é necesario modificar unha consulta ou crear índices.

26.2. Sintaxe

Para analizar unha consulta engadimos `EXPLAIN` antes da instrución SQL.

```
EXPLAIN
SELECT *
FROM alumnos
WHERE edad >= 18;
```

MariaDB non executa a consulta para devolver os seus resultados: mostra información sobre o plan de execución previsto.

26.3. Información do resultado

Entre as columnas máis importantes do resultado de `EXPLAIN` destacan as seguintes:

Columna	Significado
`id`	Identificador da consulta ou subconsulta.
`select_type`	Tipo de consulta: simple, subconsulta, unión, etc.
`table`	Tabela que se consulta.
`type`	Tipo de acceso utilizado para recuperar os rexistros.
`possible_keys`	Índices que poderían utilizarse.
`key`	Índice seleccionado polo optimizador.
`rows`	Estimación do número de filas que se deben examinar.
`Extra`	Información adicional sobre o procesamento.

26.4. Tipos de acceso

A columna `type` indica como se accede aos rexistros da táboa. Algúns valores habituais son:

Tipo	Descrición xeral
`system`	Táboa con moi poucos rexistros, equivalente a unha táboa constante.
`const`	Acceso a un único rexistro mediante unha clave única ou primaria.
`eq_ref`	Acceso mediante unha clave única para cada combinación de táboas.
`ref`	Acceso mediante un índice que pode devolver varios rexistros.
`range`	Acceso a un intervalo de valores dun índice.
`index`	Lectura do índice completo.
`ALL`	Lectura completa da táboa.

En xeral, os accesos por clave ou índice permiten localizar rexistros sen examinar toda a táboa. Un acceso `ALL` pode ser custoso cando a táboa contén moitos rexistros.

> **Importante:** un tipo de acceso `ALL` non significa necesariamente que a consulta estea mal optimizada. Nunha táboa pequena ou cando se necesita recuperar unha parte moi grande dos seus rexistros, ler a táboa completa pode ser a opción máis eficiente.

26.5. Analizar unha consulta sen índice

Supoñamos que queremos buscar os alumnos dunha idade determinada:

```
EXPLAIN
SELECT nombre, edad
FROM alumnos
WHERE edad = 20;
```

Se non existe un índice adecuado sobre `edad`, o plan pode mostrar un acceso de tipo `ALL`. Isto significa que o motor debe examinar todos os rexistros da táboa para comprobar a condición.

26.6. Analizar a mesma consulta con índice

Podemos crear un índice sobre a columna utilizada no filtro:

```
CREATE INDEX idx_alumnos_edad
ON alumnos(edad);
```

Despois volvemos executar:

```
EXPLAIN
SELECT nombre, edad
FROM alumnos
WHERE edad = 20;
```

O optimizador pode utilizar o índice para localizar os rexistros que cumpren a condición. O plan pode mostrar un tipo de acceso como `ref` ou outro adecuado, dependendo dos datos e das estatísticas.

26.7. Analizar consultas con JOIN

Tamén podemos utilizar `EXPLAIN` para estudar consultas que combinan varias táboas.

```
EXPLAIN
SELECT a.nombre,
       c.nombre AS curso
FROM alumnos a
INNER JOIN cursos c
    ON a.curso_id = c.id
WHERE c.nombre = 'DAW1';
```

O plan permite observar que táboas se consultan, en que orde e que índices se utilizan para realizar a composición.

26.8. Información adicional: Extra

A columna `Extra` pode mostrar información adicional sobre a execución. Algúns valores habituais son:

- `Using where`: aplícase unha condición de filtrado.
- `Using index`: a consulta pode obter a información necesaria directamente desde o índice.
- `Using temporary`: utilízase unha táboa temporal para procesar parte da consulta.
- `Using filesort`: é necesario realizar unha operación adicional de ordenación.

A presenza de `Using temporary` ou `Using filesort` non implica necesariamente un problema, pero pode ser interesante investigar o seu impacto en consultas con moitos rexistros.

26.9. EXPLAIN ANALYZE

Ademais de `EXPLAIN`, MariaDB permite utilizar `ANALYZE` para executar unha consulta e obter información real sobre o seu procesamento.

```
ANALYZE
SELECT nombre, edad
FROM alumnos
WHERE edad >= 18;
```

A saída inclúe información sobre o número real de filas examinadas e procesadas, ademais de estimacións do optimizador.

> **Atención:** a diferenza de `EXPLAIN`, `ANALYZE` executa a consulta. Polo tanto, debe utilizarse con precaución en instrucións que modifiquen datos.

26.10. Como identificar consultas pouco eficientes?

Ao analizar un plan de execución podemos prestar atención aos seguintes aspectos:

- Accesos `ALL` sobre táboas con moitos rexistros.
- Ausencia de índices nas columnas utilizadas frecuentemente nos filtros.
- Un número estimado de filas moi elevado.
- Operacións de ordenación ou agrupamento custosas.
- Composicións entre táboas que non utilizan adecuadamente as claves.

A partir desta información podemos probar cambios na consulta, crear índices cando estea xustificado e volver analizar o plan para comprobar se mellorou.

26.11. Exercicios de consolidación

1. Executa `EXPLAIN` sobre unha consulta que filtre alumnos pola idade.
2. Crea un índice sobre a columna `edad` e repite a análise.
3. Compara os valores das columnas `type` , `key` e `rows` .
4. Analiza cun `EXPLAIN` unha consulta que utilice `INNER JOIN` .
5. Analiza unha consulta que utilice `GROUP BY` e observa a información da columna `Extra` .
6. Compara o plan de execución antes e despois de crear un índice adecuado.

27. Práctica global: consultas SQL

27.1. Obxectivo da práctica

Nesta práctica aplicarás os coñecementos adquiridos ao longo da unidade didáctica para resolver diferentes necesidades de consulta sobre unha base de datos relacional.

Deberás utilizar consultas simples, filtros, agrupamentos, composicións entre táboas, subconsultas e operacións de combinación de resultados.

27.2. Base de datos de traballo

Utilizarase a base de datos de formación creada nas sesións anteriores, que contén as seguintes táboas:

- `cursos` : información dos cursos.
- `alumnos` : información dos alumnos e o curso ao que pertencen.
- `profesores` : información dos profesores.
- `Alumno` : datos dos alumnos no modelo de matrículas.

- `Modulo`: información dos módulos formativos.
- `Matricula`: relación entre alumnos e módulos, coas súas cualificacións.

Empregarase o esquema que corresponda a cada exercicio. Antes de comezar, comproba que as táboas conteñen datos suficientes para obter resultados significativos.

27.3. Bloque 1. Consultas simples e filtrado

1. Obtén o nome e a idade de todos os alumnos, ordenados alfabeticamente.
2. Mostra os alumnos maiores de idade.
3. Obtén os alumnos que pertencen a cursos cun número de horas comprendido entre 150 e 200.
4. Mostra os cursos que teñen máis de 150 horas, ordenados de maior a menor duración.
5. Obtén os cinco primeiros alumnos ordenados pola idade.

27.4. Bloque 2. Funcións de resumo e agrupamentos

1. Calcula o número total de alumnos.
2. Calcula a idade media dos alumnos.
3. Obtén o número de alumnos de cada curso.
4. Mostra unicamente os cursos que teñen máis de dous alumnos.
5. Calcula a idade mínima e máxima dos alumnos de cada curso.
6. Obtén os cursos nos que a idade media dos alumnos é superior á media global.

27.5. Bloque 3. Composicións entre táboas

1. Mostra o nome de cada alumno xunto co nome do curso ao que pertence.
2. Obtén os alumnos que pertencen a un curso determinado.
3. Mostra todos os cursos e os alumnos asociados, incluíndo os cursos sen alumnos.
4. Obtén os cursos que non teñen alumnos.
5. Mostra o nome dos alumnos, os módulos nos que están matriculados e as súas notas.
6. Obtén os alumnos que teñen unha nota igual ou superior a 5 nun módulo determinado.

27.6. Bloque 4. Subconsultas

1. Obtén os alumnos que teñen unha idade superior á media.
2. Mostra os cursos que teñen máis horas que a media de todos os cursos.
3. Obtén os cursos que non teñen ningún alumno mediante `NOT EXISTS`.

4. Mostra os alumnos que teñen polo menos unha matrícula aprobada.
5. Obtén os alumnos que teñen unha nota superior á súa propia media.
6. Mostra os cursos que teñen máis de dous alumnos mediante unha subconsulta con agrupamento.

27.7. Bloque 5. Combinación de resultados

1. Combina os nomes dos alumnos e dos profesores mediante `UNION`.
2. Repite a consulta utilizando `UNION ALL`.
3. Ordena alfabeticamente o resultado combinado.
4. Obtén os nomes que aparecen tanto na táboa de alumnos como na de antigos alumnos mediante `INTERSECT`, se esa táboa está dispoñible.

27.8. Bloque 6. Optimización

1. Selecciona unha consulta dos bloques anteriores que utilice unha condición de filtrado.
2. Analiza o seu plan de execución mediante `EXPLAIN`.
3. Identifica as táboas consultadas, o tipo de acceso e os índices utilizados.
4. Crea un índice sobre unha columna que se utilice frecuentemente nun filtro ou nunha composición.
5. Repite a análise e compara os resultados obtidos.
6. Explica se a modificación mellorou o plan de execución e xustifica a resposta.

27.9. Entrega da práctica

A práctica deberá incluír:

- As instrucións SQL necesarias para resolver todos os exercicios.
- Os resultados obtidos nas consultas.
- As capturas ou saídas dos plans de execución solicitados.
- Unha breve explicación das consultas que utilizan agrupamentos, composicións e subconsultas.
- Unha comparación do plan de execución antes e despois da creación do índice.

As consultas deberán estar correctamente identificadas e organizadas por bloques. Comproba que todas se executan correctamente e que os resultados corresponden co solicitado.